Name- Shreyash Khapekar

PRN - B25ET1019

Division- SY1 ENTC

Title: Implement UART communication using Data Structures in C

```
#include <stdio.h>
#define SIZE 5

typedef struct {
    char buffer[SIZE];
    int head;
    int tail;
    int count;
} CircularBuffer;

void initBuffer(CircularBuffer *cb){
    cb->head = 0;
    cb->tail = 0;
    cb->count = 0;
}

int is_buffer_full(CircularBuffer *cb){
    return cb->count == SIZE;
}

int is_buffer_empty(CircularBuffer *cb){
    return cb->count == 0;
}

int enqueue(CircularBuffer *cb, char data){
    if(is_buffer_full(cb)){
        return 0;
    }
    cb->buffer[cb->tail] = data;
    cb->tail = (cb->tail + 1) % SIZE;
    cb->count++;
    return 1;
}

char dequeue(CircularBuffer *cb){
    if(is_buffer_empty(cb)){
        return '\0';
    }
    char data = cb->buffer[cb->head];
    cb->head = (cb->head + 1) % SIZE;
    cb->count--;
```

```
    return data;
}

void uart_send(CircularBuffer *tx, char data){
    if(enqueue(tx, data)){
        printf("Byte '%c' added to TX buffer.\n", data);
    } else {
        printf("TX Buffer Overflow! Cannot send '%c'\n", data);
    }
}

void uart_transmit_hardware(CircularBuffer *tx){
    if(is_buffer_empty(tx)){
        printf("TX Buffer is empty. Nothing to transmit.\n");
    } else {
        char data = dequeue(tx);
        printf("UART Transmitted Byte: '%c'\n", data);
    }
}

void uart_receive(CircularBuffer *rx, char data){
    if(enqueue(rx, data)){
        printf("UART Received Byte: '%c' -> Stored in RX buffer.\n", data);
    } else {
        printf("RX Buffer Overflow! Dropped byte '%c'\n", data);
    }
}

void read_rx_buffer(CircularBuffer *rx){
    if(is_buffer_empty(rx)){
        printf("RX Buffer is empty.\n");
    } else {
        char data = dequeue(rx);
        printf("Application read byte from RX buffer: '%c'\n", data);
    }
}

void displayBuffer(CircularBuffer *cb, const char *name){
    if(is_buffer_empty(cb)){
        printf("%s Buffer is empty.\n", name);
        return;
    }
    printf("%s Buffer contents: ", name);
    int i, idx = cb->head;
    for(i = 0; i < cb->count; i++){
        printf("%c ", cb->buffer[idx]);
        idx = (idx + 1) % SIZE;
    }
    printf("\n");
}
```

```
int main(){
    CircularBuffer txBuf, rxBuf;
    initBuffer(&txBuf);
    initBuffer(&rxBuf);

    int choice;
    char ch;

    while(1){
        printf("\n--- UART COMMUNICATION MENU ---\n");
        printf("1. Transmit Data (Enqueue to TX Buffer)\n");
        printf("2. UART Send Hardware Byte (Dequeue TX Buffer)\n");
        printf("3. UART Receive Interrupt (Enqueue to RX Buffer)\n");
        printf("4. Read Received Data (Dequeue RX Buffer)\n");
        printf("5. Display Buffers Status\n");
        printf("6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if(choice == 1){
            printf("Enter character to send: ");
            scanf(" %c", &ch);
            uart_send(&txBuf, ch);
        }
        else if(choice == 2){
            uart_transmit_hardware(&txBuf);
        }
        else if(choice == 3){
            printf("Enter incoming character: ");
            scanf(" %c", &ch);
            uart_receive(&rxBuf, ch);
        }
        else if(choice == 4){
            read_rx_buffer(&rxBuf);
        }
        else if(choice == 5){
            displayBuffer(&txBuf, "TX");
            displayBuffer(&rxBuf, "RX");
        }
        else if(choice == 6){
            break;
        }
        else{
            printf("Invalid choice!\n");
        }
    }

    return 0;
}
```